SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

*I **RenugaDevi .B(192311111)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: RenugaDevi .B

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Testing as an Engineering Activity – Banking Application

Testing should be treated like an engineering process rather than random execution of test cases. For a banking application, the team can first identify critical functions such as login, balance enquiry, fund transfer and bill payment, and then design tests systematically from requirements.

A useful approach is to create a Requirement → Risk → Test Case → Result → Defect chain. Test cases can be designed using equivalence partitioning, boundary value analysis, decision tables and state-transition testing. Automation tools can execute repetitive regression tests, while standards such as IEEE-based documentation practices can provide consistency.

For example, in fund transfer, instead of randomly entering amounts, testers can check ₹0, negative values, minimum/maximum limits, insufficient balance, invalid beneficiary and network interruption.

This engineering-based approach increases defect detection because important combinations are deliberately covered. Ad-hoc testing can therefore be replaced with risk-based testing, model-based testing, boundary testing and automated regression testing.

2. Structured Testing Process for ATM Withdrawal

An ATM withdrawal should be tested throughout the SDLC rather than only after coding.

- Requirement stage: identify rules such as valid PIN, sufficient balance and withdrawal limits.
- Design stage: prepare scenarios for card insertion, authentication and cash dispensing.
- Development stage: developers perform unit testing on PIN validation and balance calculation.
- Integration stage: verify communication between ATM, bank server and cash dispenser.
- System testing: test the complete withdrawal process.
- Regression testing: repeat important tests after modifications.
- Acceptance testing: verify whether the ATM behaves according to customer and banking requirements.

For example, withdrawing ₹5,000 with sufficient balance should succeed, while withdrawing more than the daily limit should be rejected.A defined process prevents testers from forgetting important

stages and produces repeatable results. The expected outcome is fewer escaped defects, predictable testing, faster diagnosis and improved ATM reliability.

3. Error, Defect and Failure in Login/Transaction

These three terms describe different points in the same problem.

Suppose a programmer writes: if(password =

enteredPassword) when the intended operation was

comparison using ==.

- Error: the programmer makes an incorrect coding decision.
- Defect: the incorrect comparison exists inside the software.
- Failure: when a genuine user enters the wrong password and the system still allows login.

The relationship can be viewed as:

Human mistake → Defect in software → Incorrect execution → Failure observed by user

Similarly, if a transaction displays ₹10,000 while the account actually contains ₹8,000, the incorrect calculation may be a defect and the wrong displayed balance is the resulting failure.Confusing these terms produces poor defect reports such as “login is not working.” A better report should mention actual result, expected result, reproduction steps, environment, severity and evidence. This gives developers enough information to locate the underlying defect.

4. Defect Clustering and Test Prioritization

Defects are rarely distributed equally across a system. A small number of modules often contain a large percentage of defects. This is known as defect clustering.

Suppose an e-commerce application has 10 modules, but payment, checkout and discount calculation contain 70% of previously reported defects. Testing effort should not be divided equally among all modules.

A priority model can be:

Priority = Business Risk × Defect History × Usage Frequency Therefore:

1. Payment processing – very high priority
2. Checkout – very high priority
3. Login – high priority
4. Product search – medium priority
5. About Us page – low priority

Other testing principles, such as exhaustive testing being impossible and pesticide paradox, also encourage continuously improving test cases.

For example, if previous releases repeatedly failed during festival discounts, those scenarios should receive additional regression tests before the next sale.

This concentrates effort where failures are most likely, improving testing efficiency without simply increasing the number of test cases.

5. Using a Defect Repository Effectively

A defect repository should not become a digital storage box containing old bug reports. It should act as a knowledge base for future testing.

Defects can be classified according to:

- Functional defects
- Interface defects
- Performance defects
- Security defects
- Data-related defects
- Compatibility defects
- Logic defects

Suppose three releases show different bugs related to incorrect tax calculations. Repository analysis may reveal that all originated from the same pricing component.

Testers can then create a permanent regression suite around tax calculation and related boundary conditions.

The repository can also record module, root cause, severity, frequency, release and resolution. Historical patterns can therefore influence future test planning.

To prevent recurrence, teams should perform root-cause analysis and convert important historical defects into regression tests. The expected improvement is less defect repetition, stronger regression coverage and preservation of testing knowledge across releases.

6. Mobile Banking Fund Transfer Crashes

A fund-transfer crash should not be investigated by simply repeating the same successful transaction. Testing should reproduce the failure under different conditions.

The tester can vary:

- Transfer amount
- Account balance
- Network strength
- Beneficiary status
- Transaction frequency
- Device type
- Operating-system version
- App background/foreground state

For example, the application may work correctly on Wi-Fi but crash when mobile connectivity changes from 4G to 5G during transaction confirmation.

Logs, crash reports and transaction IDs can help connect the visible failure with its root cause.

A systematic process would be:

Reproduce → isolate condition → collect evidence → classify defect → fix → retest → regression test.

Stress, interruption, compatibility and negative testing should be added to the normal functional suite. This prevents the team from testing only the “happy path” and improves stability in real-world banking conditions.

7. Testing vs Debugging

Testing and debugging are related, but they answer different questions.

Testing asks:
"Can we demonstrate that the software behaves incorrectly?"

Debugging asks:
"Why did it behave incorrectly, and where should the code be corrected?" Consider

an ATM dispensing ₹2,000 when ₹1,000 was requested.

The tester reproduces the problem, records the expected and actual results and raises a defect. The developer then debugs the withdrawal calculation, identifies the incorrect variable or calculation and fixes it.A weak process occurs when testers themselves start modifying code without following the defect process, or developers assume that successful debugging means the entire system has been tested.

A better cycle is:

Test → Report → Debug → Fix → Retest → Regression Test → Close
This separation provides accountability while allowing testing and debugging to work together.

8. Cross-Browser Testing for an E-Commerce Platform

Browser inconsistency can occur because browsers use different rendering engines, JavaScript implementations, CSS support and security policies.

Instead of manually testing every page randomly, create a browser compatibility matrix.

Function	**Chrome**	**Edge**	**Firefox**	**Safari**
Login	✓	✓	✓	✓

Product filter	✓	✓	✓	✓
Cart	✓	✓	✓	✓
Payment	✓	✓	✓	✓

Important user journeys should receive priority: login → search → product selection → cart → payment.

Tools such as Selenium or Playwright can automate repeated browser tests, while cloud browser platforms can provide access to different browser/OS combinations.

Defects should be reported with the browser version, operating system, screen size, steps and screenshot/video. This makes compatibility defects reproducible instead of treating them as unexplained UI problems.

9. Classification of Payroll Logic Defects

A salary calculation error is not simply “a calculation bug.” It can be classified according to its origin and behavior.

For example, assume:

Net Salary = Basic Salary + Allowances − Deductions

If the programmer accidentally subtracts allowances instead of adding them, this is a logic defect.

Other payroll defects may include:

- Calculation defect: incorrect arithmetic

- Boundary defect: wrong result at salary/tax limits
- Data defect: incorrect employee information
- Requirement defect: wrong interpretation of tax rules
- Integration defect: incorrect connection with attendance data

Classification helps testers select appropriate techniques. Logic defects require calculation-based tests, while boundary defects need boundary value analysis.

A useful prevention strategy is to maintain independent expected-value calculations and automate salary regression tests. This reduces the possibility that a small code modification silently changes employee salaries.

10. Replacing Random Testing with Designed Testing

Random testing may occasionally discover defects, but it does not prove that important requirements have been covered.

A better process begins by converting requirements into test conditions and then into designed test cases.

For example, for a password field:

- Empty password
- Minimum valid length
- Maximum valid length
- One character below the limit
- One character above the limit
- Special characters
- Incorrect password
- Correct password

Techniques such as equivalence partitioning, boundary value analysis, decision tables and state-transition testing provide systematic coverage.

Test cases should also have unique IDs, preconditions, input data, expected results and traceability to requirements.

The improvement is not simply “more test cases.” It is better-selected test cases, meaning fewer meaningless tests and greater probability of finding important defects.

11. Data Inconsistency in a Healthcare Application

Healthcare data must remain consistent because the same patient information may be used by registration, appointment, billing and medical-record modules.

Suppose a patient's date of birth is updated in registration but the billing module continues showing the old value. This may indicate an integration, database synchronization or transaction-management defect.

Testing should verify:

Input → Database → Service/API → Receiving Module → Display

Testers can use the same patient record across modules and compare values at each stage. They should also test simultaneous updates, failed transactions and rollback scenarios.

A defect repository should classify such problems by module, data field, root cause and affected interfaces.

Clear reports containing patient-test ID, old value, new value, expected synchronization behavior and evidence make investigation easier. Systematic data-integrity testing therefore prevents inconsistent information from spreading across the application.

12. Repeated Defects Across Releases

If the same defects repeatedly appear, the problem may not be insufficient testing—it may be failure to learn from previous testing.

A defect repository can be analyzed like historical data.

For example:

Defect Pattern	Frequency	Action
Login timeout	8	Add regression tests
Payment failure	12	Strengthen integration tests
Discount calculation	7	Add boundary tests
Mobile UI issue	5	Expand device testing

This transforms the repository from a reporting system into a test-planning system.

Testers can identify recurring modules, defect categories, root causes and releases with unusually high defect counts. High-frequency defects should become permanent regression scenarios.

The result is a feedback loop:

Old defect → Repository → Pattern analysis → New test → Prevention → Fewer repeated defects

13. Learning System Failure During Peak Usage

A learning platform may work perfectly with 100 students but fail when thousands access it simultaneously. This indicates a gap between functional coverage and realistic usage coverage.

Test planning should consider:

- Concurrent users
- Login bursts
- Simultaneous video access
- Quiz submission peaks
- Database load
- Network delays

- Session expiration

Performance testing can gradually increase users to identify the point where response time becomes unacceptable.

For example, if the system is tested only with 50 users but the actual examination requires 5,000 students to submit answers within five minutes, the original test plan is incomplete.

Load, stress, spike and endurance testing should therefore be included. Better coverage exposes capacity limitations before deployment, improving reliability during actual examinations.

14. Early Tester Involvement in SDLC

A tester should not enter the project only after developers announce, “The software is ready.” During requirements analysis, testers can identify ambiguous or missing conditions. During design, they can identify difficult-to-test interfaces. During development, they can prepare test data and automation.

For example, suppose a requirement says:

> “The system should lock the account after multiple incorrect passwords.”

A tester can immediately ask: What does “multiple” mean—3, 5 or 10 attempts? Does a successful login reset the counter?

Resolving this ambiguity before coding prevents later defects.

This approach is called shift-left testing. It encourages developers, testers, analysts and business stakeholders to discuss quality continuously rather than treating testing as a final-stage activity.

15. Incomplete Test Coverage

A system may pass hundreds of test cases and still fail because the tests cover only normal conditions.

Suppose an online payment system has been tested for:

- Valid card
- Correct OTP

- Sufficient balance

But no tests exist for:

- Expired card
- Wrong OTP
- Network loss
- Duplicate payment
- Session timeout
- Payment gateway failure

The issue is not necessarily the number of tests but the areas they represent.

A coverage matrix can map requirements to test cases. Risk-based prioritization can then identify high-impact gaps.

Boundary value analysis, negative testing, equivalence partitioning, decision tables and exploratory testing can strengthen coverage.

The goal is to make important conditions, states, inputs and user journeys visible in the test design.

16. Improving Defect Reporting in a Logistics Application

A defect report should allow another person to reproduce the problem without asking the tester for additional explanations.

Instead of writing:

> "Tracking is not working."

A structured report should contain:

Defect ID → Title → Environment → Preconditions → Steps → Expected Result → Actual Result → Severity → Priority → Evidence

For example, a delivery-tracking defect might state that after entering a valid tracking number, the application continues displaying "Shipment not found" although the backend contains the shipment.

Clear terminology also matters. A defect is the flaw in the software; the incorrect behavior observed during execution is the failure.

Consistent reporting standards reduce communication gaps between tester, developer and project manager. Faster reproduction means faster correction and verification.

17. Process-Based Testing Across Modules

When each module is tested according to a different personal method, results become difficult to compare. The solution is to establish a common testing workflow.

A standard process can be:

Requirement analysis → Test planning → Test design → Test data preparation → Execution → Defect reporting → Retesting → Regression → Closure

Each module can still have its own specialized tests, but the basic process remains unchanged.

For example, in an e-commerce system, login, cart and payment teams can use the same defect severity scale, test-case format and execution status.

Templates, checklists, common entry/exit criteria and peer reviews can maintain consistency.

The benefit is **repeatability**: a new tester joining the project can follow the same process instead of learning an individual's testing style.

18. Deployment Failure Due to Edge Cases

Edge cases are often missed because testers concentrate on values that represent normal users.

Suppose a ticket-booking system accepts seats numbered 1–100. Testing only seats 20, 40 and 60 does not validate the boundaries.

Boundary testing should include:

0, 1, 2, 99, 100, 101

Similarly, for a transaction limit of ₹1–₹50,000, testers should specifically test ₹0, ₹1, ₹2, ₹49,999, ₹50,000 and ₹50,001.

Edge cases should be identified directly from requirements and business rules before execution. They should then become permanent regression tests.
This is especially important because boundary defects can remain hidden during ordinary testing but become highly visible after deployment.

19. Defect Repository for Poor Documentation

A defect repository should provide a memory for the testing team.

Every defect should contain enough information to answer:

- What happened?
- Where did it happen?
- When was it found?
- Who reported it?
- How serious is it? ● Why did it occur?
- How was it fixed?
- Has it been retested?

For example, instead of storing “Payment bug – fixed,” the repository should contain the affected release, payment method, reproduction steps, root cause and verification status.

Structured repositories allow testers to identify patterns such as “most critical defects occur in payment integration.”

Useful fields include severity, priority, module, defect type, root cause, status, release and resolution.

Good documentation therefore supports not only tracking but also future test planning and preventive action.

20. Banking Transaction Failures – Redesigned Testing Approach

For repeated transaction failures, I would redesign testing around the transaction lifecycle rather than testing isolated screens.

The lifecycle can be represented as:

Login → Select beneficiary → Enter amount → Validate → Authorize → Debit → Server confirmation → Receipt → Balance update

Each transition should have normal, abnormal and interruption scenarios.

For example, testers should verify what happens if the network disconnects immediately after the amount is debited. The most dangerous defect would be a situation where the user is charged but the transaction is shown as failed.

Testing should therefore include:

- Functional testing
- Negative testing
- Boundary testing
- API/integration testing
- Security testing
- Recovery testing
- Concurrency testing
- Regression automation

Risk-based prioritization should place money movement above low-risk features such as profile customization.

The redesigned process creates stronger control over transaction states and significantly reduces the chance of financial inconsistencies reaching customers.